

1 D.1 — Apprehender la Notion du Conteneur

1.1 Definition

Conteneur

Un **conteneur** est une unite d'execution legere et isolee qui embarque une application ainsi que **toutes ses dependances** (bibliothèques, fichiers de configuration, variables d'environnement) dans un paquet portable.

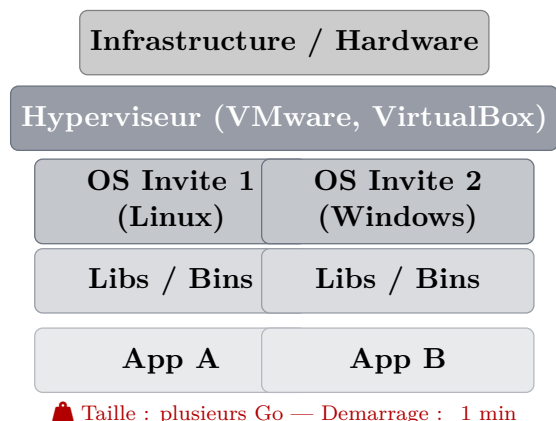
Il s'exécute de façon identique **sur n'importe quelle machine** qui possède un moteur de conteneurisation, indépendamment du système d'exploitation hôte.

Analogie : le conteneur de transport

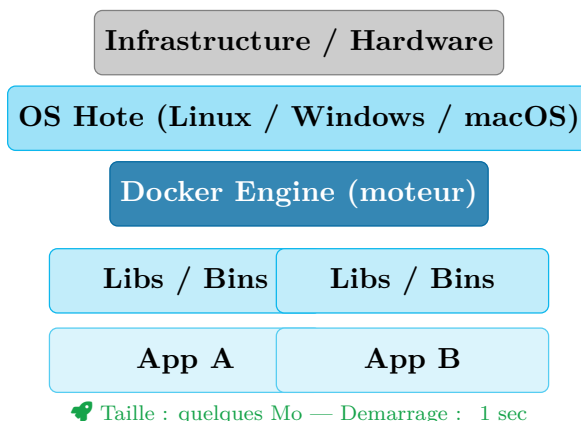
Pensez au conteneur de fret maritime : peu importe ce qu'il contient, il peut être chargé sur n'importe quel navire, train ou camion. De la même façon, un conteneur logiciel peut être exécuté sur n'importe quel serveur sans modifier son contenu.

1.2 Difference entre Machine Virtuelle et Conteneur

Machine Virtuelle (VM)



Conteneur Docker

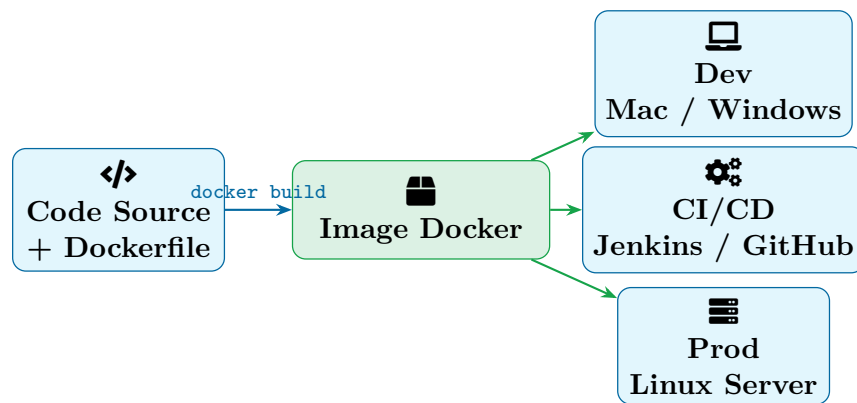


Critere	Machine Virtuelle	Conteneur Docker
OS	Systeme d'exploitation complet inclus	Partage le noyau de l'OS hôte
Taille	Plusieurs gigaoctets	Quelques megaoctets
Demarrage	1 a 5 minutes	Moins d'une seconde
Isolation	Isolation complete (hardware)	Isolation au niveau processus
Portabilite	Limitee (hyperviseur specifique)	Tres haute (tout systeme Docker)
Ressources	Consommation elevee	Tres legere

1.3 Avantages des Conteneurs

Pourquoi utiliser Docker ?

1. **Portabilite** : “*Build once, run anywhere*” — un conteneur fonctionne de facon identique en dev, test et production.
2. **Isolation** : chaque service tourne dans son propre environnement, sans conflit de versions ou de dependances.
3. **Rapidite** : demarrage en quelques secondes, deploiement instantane.
4. **Reproductibilite** : l’environnement est decrit dans un fichier texte (Dockerfile), versionnable avec Git.
5. **Scalabilite** : il est trivial de lancer 10 instances du meme conteneur derriere un load balancer.
6. **Ecosysteme riche** : Docker Hub offre des milliers d’images officielles (Node.js, MongoDB, Nginx, RabbitMQ...).



Comportement identique partout

2 D.2 — Prise en Main de Docker

2.1 Installation de Docker Desktop

Docker Desktop

Docker Desktop est l'application officielle qui installe en un clic : le moteur Docker, Docker Compose, et une interface graphique de gestion. Elle est disponible pour **Windows**, **macOS** et **Linux**.

2.1.1 Procédure d'installation

1. Aller sur <https://www.docker.com/products/docker-desktop>
2. Télécharger la version correspondant a votre OS
3. Executer l'installateur et suivre les instructions
4. Redemarrer le poste si demande
5. Lancer Docker Desktop et attendre que le moteur soit actif (● vert en bas a gauche)

```
# Verifier la version de Docker
docker --version
# Docker version 25.0.3, build 4debf41

# Verifier que Docker fonctionne
docker run hello-world
# Affiche : Hello from Docker!

# Verifier Docker Compose
docker compose version
# Docker Compose version v2.24.5
```

Listing 1: Verification de l'installation

✓ Test rapide : premier conteneur

La commande `docker run hello-world` telecharge automatiquement l'image `hello-world` depuis Docker Hub, cree un conteneur et affiche un message de confirmation. Si vous voyez ce message, Docker est correctement installe.

2.2 Terminologies Docker

2.2.1 Image Docker

Image

Une **image** Docker est un modele **en lecture seule** qui contient le systeme de fichiers et les instructions necessaires pour creer un conteneur. C'est l'equivalent d'une "recette" ou d'un "moule".

Une image est construite en couches (**layers**). Chaque instruction du Dockerfile

ajoute une couche immuable.

2.2.2 Conteneur

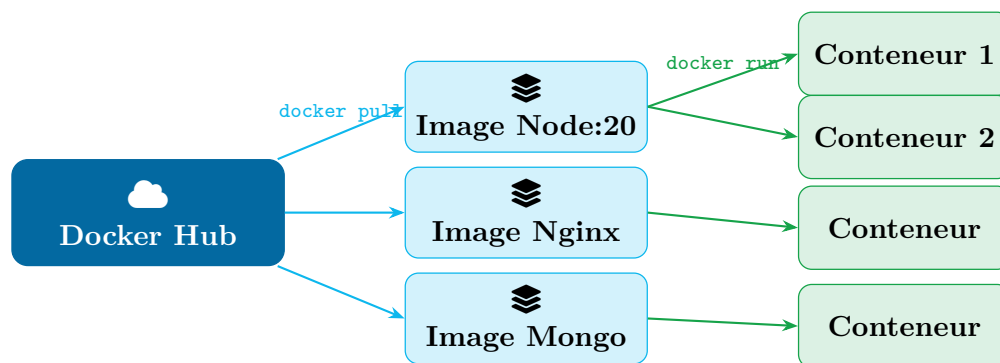
Conteneur

Un **conteneur** est une **instance en cours d'exécution** d'une image. On peut créer plusieurs conteneurs à partir de la même image, exactement comme on peut imprimer plusieurs copies d'un même document.

2.2.3 Docker Hub

Docker Hub

Docker Hub (<https://hub.docker.com>) est le registre public officiel d'images Docker. Il contient des milliers d'images officielles maintenues par les éditeurs (Node.js, MongoDB, Nginx, Redis, RabbitMQ...) et des images communautaires.



docker pull = télécharger une image docker run = créer + démarrer un conteneur

2.2.4 Commandes Docker essentielles

Commande	Description
<code>docker pull image</code>	Télécharger une image depuis Docker Hub
<code>docker images</code>	Lister toutes les images locales
<code>docker run image</code>	Créer et démarrer un conteneur
<code>docker run -d image</code>	Démarrer en mode détaché (arrière-plan)
<code>docker run -p 3000:3000 img</code>	Mapper le port hôte:conteneur
<code>docker run -name nom img</code>	Nommer le conteneur
<code>docker ps</code>	Lister les conteneurs en cours d'exécution
<code>docker ps -a</code>	Lister tous les conteneurs (même arrêtés)
<code>docker stop id</code>	Arrêter un conteneur
<code>docker start id</code>	Redémarrer un conteneur arrêté
<code>docker rm id</code>	Supprimer un conteneur arrêté
<code>docker rmi image</code>	Supprimer une image locale
<code>docker logs id</code>	Voir les logs d'un conteneur
<code>docker exec -it id bash</code>	Entrer dans un conteneur en cours
<code>docker build -t nom .</code>	Construire une image depuis le Dockerfile

2.3 Création d'un Dockerfile pour une Application Node.js

2.3.1 Qu'est-ce qu'un Dockerfile ?

Dockerfile

Un **Dockerfile** est un fichier texte qui contient les instructions pour construire une image Docker. Chaque instruction crée une nouvelle couche dans l'image.

Instruction	Role
FROM	Image de base (point de départ)
WORKDIR	Définir le répertoire de travail dans le conteneur
COPY	Copier des fichiers depuis la machine hôte vers l'image
RUN	Exécuter une commande pendant la construction de l'image
EXPOSE	Déclarer le port sur lequel l'application écoute
ENV	Définir des variables d'environnement
CMD	Commande par défaut au démarrage du conteneur

2.3.2 Application Node.js a conteneuriser

```

1  const express = require('express');
2  const app     = express();
3
4  app.use(express.json());
5
6  app.get('/', (req, res) => {
7      res.json({
8          message: 'API en ligne',
9          env: process.env.NODE_ENV || 'development'
10     });
11 });
12
13 app.get('/health', (req, res) => {
14     res.json({ status: 'OK' });
15 });
16
17 const PORT = process.env.PORT || 3000;
18 app.listen(PORT, () => {
19     console.log('Serveur démarre sur le port ${PORT}');
20 });

```

Listing 2: app.js — Application Express simple

```

1  mon-api/
2    |-- app.js
3    |-- package.json
4    |-- .dockerignore      # Fichiers a exclure de l'image
5    |-- Dockerfile

```

Listing 3: Structure du projet a conteneuriser

```

1  node_modules
2  npm-debug.log
3  .git
4  .env
5  *.md

```

Listing 4: .dockerignore — Exclure les fichiers inutiles

2.3.3 Ecriture du Dockerfile

```

1  # =====
2  # ETAPE 1 : Image de base
3  # =====
4  # On part de l'image officielle Node.js version 20 (LTS)
5  # La variante "alpine" est legere (~5 Mo contre ~300 Mo)
6  FROM node:20-alpine
7
8  # =====
9  # ETAPE 2 : Repertoire de travail
10 # =====
11 # Tous les fichiers seront places dans /app dans le conteneur

```

```

12 WORKDIR /app
13
14 # =====
15 # ETAPE 3 : Installer les dependances
16 # =====
17 # On copie d'abord package.json AVANT le code source.
18 # Avantage : Docker met en cache cette couche si package.json
19 # n'a pas change --> npm install ne se relance pas a chaque build
20
21 COPY package*.json ./
22
23 RUN npm install --only=production
24
25 # =====
26 # ETAPE 4 : Copier le code source
27 # =====
28 COPY . .
29
30 # =====
31 # ETAPE 5 : Variables d'environnement
32 # =====
33 ENV NODE_ENV=production
34 ENV PORT=3000
35
36 # =====
37 # ETAPE 6 : Exposer le port
38 # =====
39 # Indication documentaire du port utilise
40 EXPOSE 3000
41
42 # =====
43 # ETAPE 7 : Commande de demarrage
44 # =====
45 CMD ["node", "app.js"]

```

Listing 5: Dockerfile — Application Node.js

```

# Construire l'image (le point "." = repertoire courant)
docker build -t mon-api:1.0 .

# Verifier que l'image est bien cree
docker images

# Lancer le conteneur
docker run -d \
  --name mon-api-ctn \
  -p 3000:3000 \
  -e NODE_ENV=production \
  mon-api:1.0

# Tester l'API
curl http://localhost:3000
# {"message": "API en ligne", "env": "production"}

```

```
# Voir les logs  
docker logs mon-api-ctn
```

Listing 6: Construire et lancer l'image